Part A: Making the Fronthaul Lighter and the DU Faster

Structure-aware IQ compression, RMT-guided warm-start complex inversion, and a fused C/AVX-512 RZF precoder kernel for O-RAN 7-2x

Mayank Yadav Harshvardhan Choudhary

B.Tech. Project — Final Evaluation

November 2026

Outline

1. Motivation and Contributions
2. IQ Compression and Adaptive Split
3. Matrix Inversion Benchmark
4. Complex Matrix Inversion (RMT-WS)
5. Kernel Programming
6. Fronthaul Impact and Summary

Outline

1. Motivation and Contributions
2. IQ Compression and Adaptive Split
3. Matrix Inversion Benchmark
4. Complex Matrix Inversion (RMT-WS)
5. Kernel Programming
6. Fronthaul Impact and Summary

Why Part A? Two bottlenecks of a 5G/6G O-RAN cell

1. The fronthaul is huge

- Raw 64-antenna cell: ≈ 15.4 Gbit/s
- DU-side precoding, 100 MHz, 64 streams: **118 Gbit/s**
- O-RAN BFP only shortens numbers: $\approx 4\times$

$\Rightarrow$ compression that exploits channel structure

2. The DU has 0.5 ms per slot

- RZF per PRB: $G = HH^H + \alpha I$, $W = H^H G^{-1}$
- 273 PRBs $\times$ complex $K \times K$ inverse every slot
- numpy + LAPACK: **3.2 ms** for $K = 16$ ($6\times$ over budget)

$\Rightarrow$ fast, accurate complex inversion

Our answer: compress the data with its own structure, and invert the complex channel matrices with an RMT-guided method running on hand-optimised kernels.

Part A attacks both bottlenecks: fewer bits on the link, and a precoder that fits in the slot.

Our Part A contributions at a glance

Pilot-sketch RAS-BFP

Up to **13.9**× at the 256-QAM EVM limit, **2–4.5**× more than O-RAN BFP; C encoder **8–22**× faster than numpy

ACAFS

Rank-adaptive functional split: **55–75 %** bandwidth cut at high SNR

RMT-WS inversion

Complex Hermitian Grams: **6 GEMMs** vs > 10 for Neumann-type methods

Warm-start tracking

2.7× fewer GEMMs, **30**× lower loss at pedestrian speed; never diverges at 120 km/h

Fused C/AVX-512 kernel

Whole RZF precoder **10–15**× faster than numpy+LAPACK; fits the 0.5 ms slot

RU-side precoding

Delay-domain weights: **2.4**× less fronthaul at $\geq 96\,\%$ of exact RZF, any delay spread

From compression theory to a slot-feasible precoder: a complete DU-side toolchain.

Outline

1 Motivation and Contributions

2 IQ Compression and Adaptive Split

3 Matrix Inversion Benchmark

4 Complex Matrix Inversion (RMT-WS)

5 Kernel Programming

6 Fronthaul Impact and Summary

Structure-aware IQ compression

$$Y = H \odot X + W \in \mathbb{C}^{M \times N}, \quad M = 64 \text{ antennas}, N = 1200 \text{ subcarriers, 3GPP TDL-A}$$

CSEE — delay-domain sparse encoder

1. IFFT along subcarriers $\rightarrow$ delay domain
2. Keep top-K taps (channel is sparse in delay)
3. Per-antenna BFP; FFT back at the receiver

Theorem 1: $\mathrm{NMSE} \leq \dfrac{\sigma_{\mathrm{tail}}^2}{\sigma_Y^2} + \dfrac{\Delta^2}{12}\dfrac{K}{MN\sigma_Y^2}$

Complexity $O(MN \log N)$

RAS-BFP — randomised sketch + QR

1. SRHT sketch $Z = \Omega Y \in \mathbb{C}^{r \times N}$
2. Thin QR $Z^H = QR$; project $L = YQ$
3. Send BFP(L), BFP(Q); $\quad Y \approx LQ^H$

Theorem 2:
$\|Y - LQ^H\|_F^2 \leq (1 + \varepsilon_r)\|Y - Y_r\|_F^2$ w.h.p.
Theorem 4: $r^\star = \lfloor \sqrt{B_{\mathrm{total}}/(b(M+N))} \rceil$, $O(rMN)$

Both encoders come with provable error bounds and lower complexity than SVD.

Compression at equal detection quality

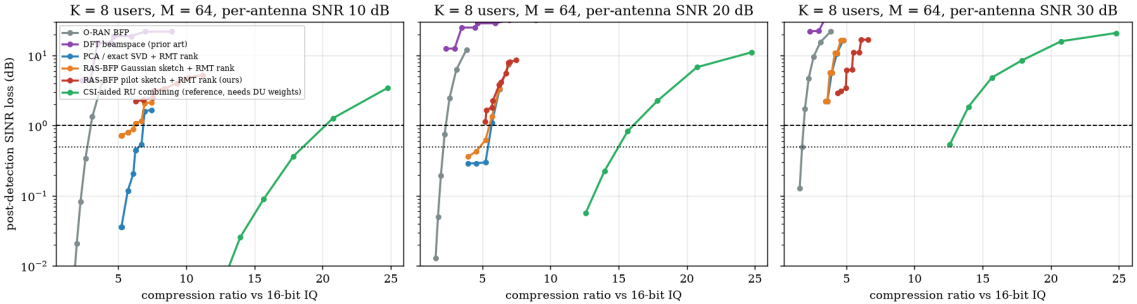
Uplink 7-2x, $M = 64$, 24 PRBs, MMSE detection, 256-QAM. Compression ratio vs 16-bit I/Q at the **3GPP 256-QAM EVM limit (3.5 %)**, all side information counted

K, SNR	BFP	DFT	SVD	**Ours**
4, 20 dB	3.1×	2.3×	13.8×	**13.9×**
4, 30 dB	3.1×	3.0×	9.2×	**11.4×**
8, 20 dB	3.1×	—	6.9×	**7.0×**
8, 30 dB	3.1×	—	4.8×	**6.6×**

- **Ours:** DMRS-pilot sketch + Marchenko–Pastur noise-edge rank + BFP basis/coefficients
- Per-PRB rank adapts to frequency selectivity (16–26 $> K$)
- **2–4.5×** more than O-RAN BFP at the same EVM; beats exact SVD at high SNR

Measured where it matters — at the EVM limit — our encoder compresses 2–4.5× more than the standard.

Compression ratio versus post-detection SINR loss



Our pilot-sketch encoder (red) reaches more compression than O-RAN BFP (grey) and DFT beamspace (purple) at every loss level.

A fast encoder that adapts its basis refresh to user speed

C/OpenMP encoder (`kernels/iq_kernels.c`), 273 PRBs, ms/slot

Encoder	1 thr	4 thr
numpy pilot sketch	80.6	84.2
C, basis refreshed every slot	34.9	10.4
C, 25 % of PRBs refreshed	**12.7**	**3.8**

- Batched LAPACK `cheevd`, register-blocked projection, vectorised BFP
- Projection is 0.37 GFLOP/slot: a natural fit for an FPGA/GPU offload

Out-of-sample certificate: refresh a PRB's basis only when a validation symbol leaves the stored subspace

Speed	**Certificate**	Every 8 slots
3 km/h	**18–43 % refreshed, 7.4×**	7.5×
30 km/h	**100 %, 5.7×, 2.6 dB**	33 dB loss
120 km/h	**100 %, 5.7×, 2.6 dB**	fails

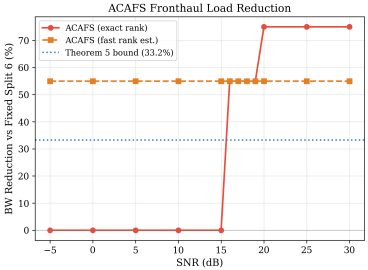
Static users get a cheaper, higher-ratio encoder; fast users are never left with a stale basis.

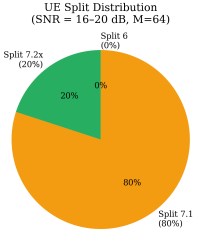
ACAFS: rank-adaptive functional split

Per-UE rank fraction r/M picks the O-RAN split:

Rank fraction r/M	Split
< 0.15	7.2x
$0.15 - 0.55$	7.1
≥ 0.55	6

- **Theorem 5:** closed-form bandwidth saving vs a fixed Split 6
- $\approx 33\,\%$ saving for uniformly spread ranks
- **55–75 %** saving at high SNR (rank collapse)
- Decision per UE, from the rank the encoder already computes





The split adapts to the channel instead of being fixed at deployment.

Outline

1. Motivation and Contributions
2. IQ Compression and Adaptive Split
3. Matrix Inversion Benchmark
4. Complex Matrix Inversion (RMT-WS)
5. Kernel Programming
6. Fronthaul Impact and Summary

Matrix-inversion benchmark: why and how

Why inversion?

MMSE equalisation, RZF precoding, channel estimation and our encoders need an inverse **every 0.5 ms slot**.

Dataset (`matinv_bench/`)

- $A = G/\sqrt{n} + 2I, \quad G_{ij} \sim \mathcal{N}(0,1)$, float32
- $n \in \{10, 100, 500\}$, 1000 matrices each
- 80 % train (learned models), timing on 200 held-out
- Per matrix, warm-up, CPU only

Existing methods (6)

LAPACK `getri` (numpy), LU (scipy), QR, SVD, Gauss–Jordan, Newton–Schulz

Our learned inverters (3)

- **InverseNet-MLP:** direct regression $A \mapsto A^{-1}$
- **InverseNet-NS:** unrolled Newton–Schulz, 8 steps, learned step coefficients
- **InverseNet-Ultra:** learned high-order iteration

9 methods, one held-out dataset, identical timing protocol.

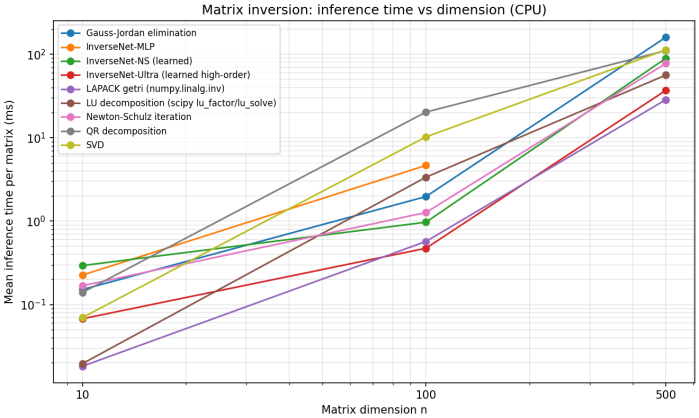
Benchmark results: InverseNet-Ultra vs existing methods

Method	$n = 100$ Mean	$n = 100$ p95	$n = 100$ Rel. err	$n = 500$ Mean	$n = 500$ p95	$n = 500$ Rel. err
InverseNet-Ultra (ours)	**0.47 ms**	**0.57 ms**	$1.4 \cdot 10^{-6}$	37.0 ms	52.7 ms	$1.4 \cdot 10^{-6}$
LAPACK `getri` (numpy)	0.57 ms	1.13 ms	$4.8 \cdot 10^{-8}$	28.6 ms	45.1 ms	$4.8 \cdot 10^{-8}$
InverseNet-NS (ours)	0.97 ms	1.06 ms	$4.0 \cdot 10^{-4}$	89.6 ms	100.9 ms	$7.3 \cdot 10^{-3}$
Newton–Schulz	1.27 ms	1.71 ms	$2.0 \cdot 10^{-7}$	78.0 ms	122.7 ms	$3.5 \cdot 10^{-7}$
Gauss–Jordan	1.97 ms	3.40 ms	$3.0 \cdot 10^{-7}$	160.3 ms	166.1 ms	$6.9 \cdot 10^{-7}$
LU (scipy)	3.36 ms	6.09 ms	$1.3 \cdot 10^{-7}$	56.3 ms	87.1 ms	$2.0 \cdot 10^{-7}$
SVD	10.2 ms	20.8 ms	$1.4 \cdot 10^{-7}$	113.0 ms	147.3 ms	$2.5 \cdot 10^{-7}$
QR	20.3 ms	64.5 ms	$3.2 \cdot 10^{-7}$	110.8 ms	153.0 ms	$3.2 \cdot 10^{-7}$

- $n = 100$: **lowest mean** latency of all 9 methods and **2× lower p95** than LAPACK (0.57 vs 1.13 ms)
- **2.7×** faster than classical Newton–Schulz, **7×** faster than LU (scipy), **22–43×** faster than SVD / QR
- $n = 500$: faster than LU, Newton–Schulz, QR, SVD and Gauss–Jordan (1.5–4.3×)

A learned iteration with 10^{-6} accuracy and the most predictable latency at $n = 100$.

Inference time vs matrix dimension



- **Direct neural inverse fails:** InverseNet-MLP has $\approx 52\,\%$ error and $> 2 \cdot 10^9$ parameters at $n = 500$
- **Learned iteration works:** keep the Newton–Schulz structure, learn only coefficients $\Rightarrow 10^{-6}$ error
- Fixed number of batched matmuls: flat, predictable latency; GPU / NPU friendly

Keep the mathematical structure, learn only what is uncertain — the idea behind RMT-WS.

Outline

1. Motivation and Contributions
2. IQ Compression and Adaptive Split
3. Matrix Inversion Benchmark
4. Complex Matrix Inversion (RMT-WS)
5. Kernel Programming
6. Fronthaul Impact and Summary

The real DU workload: complex RZF precoding

Realistic complex channels (`cmatinv/`)

- 3GPP TR 38.901 UMa NLOS path loss + shadowing (near-far)
- TDL-A 23-tap profile over 100 MHz
- Per-user angular-spread spatial correlation
- Rician line-of-sight option
- Clarke/Jakes aging $\rho = J_0(2\pi f_D T)$: 3–120 km/h
- $M = 64/128$, $K = 8/16/32$, 273 PRBs, 30 kHz, 3.5 GHz

Per PRB, every 0.5 ms slot

$$G = HH^H + \alpha I \in \mathbb{C}^{K \times K}$$

(complex Hermitian)

$$W = H^H G^{-1} \in \mathbb{C}^{M \times K}$$

$\times$ 273 PRBs inside one slot

Our question: can the whole complex precoder run inside the slot on a CPU DU, and can inversion exploit the channel's statistics?

RMT-WS: random-matrix-guided warm-start inversion

1. **Equilibrate** $\hat{G} = D^{-1/2} G D^{-1/2}$ — removes the near-far spread
2. **Eigensolve-free spectral interval** (Marchenko–Pastur + moment match):

$$v = \|\hat{G}\|_F^2/K - 1, \qquad [a, b] = 1 + v/\bar{s} \mp 2\sqrt{v} \quad (K^2 \text{ work})$$

3. **Cold start:** Chebyshev polynomial on $[a, b]$, then Newton–Schulz $X \leftarrow X(2I - \hat{G}X)$ (quadratic, pure GEMMs)
4. **Tracking:** next slot starts from the previous inverse
5. **Residual certificate** $\|(I - \hat{G}X)z\|/\|z\|$ picks the Newton steps per PRB, or falls back to a cold start — always safe

Why it fits the DU

- ✓ Adapts to correlated and near-far channels automatically
- ✓ No eigen-decomposition
- ✓ Only matrix multiplies: GPU / NPU / tensor-core friendly
- ✓ Cost adapts to user speed
- ✓ Built-in accuracy certificate

Random-matrix theory gives the inverse a head start; the certificate keeps it safe.

Cold-start inversion: fewer GEMMs

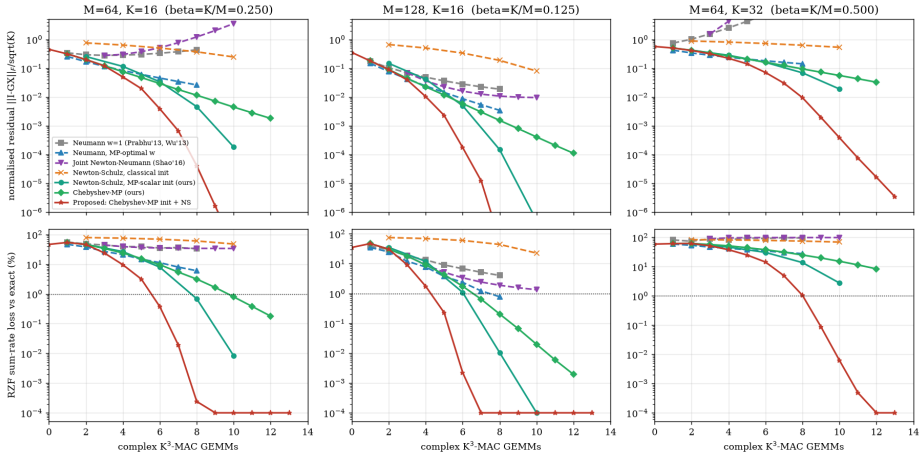
Complex GEMMs (K^3 MACs) needed for $\leq 1\,\%$ RZF sum-rate loss

Method	64/16	128/16	64/32
Neumann (Wu / Prabhu)	> 8	> 8	> 8
Joint Newton–Neumann (Shao)	> 10	> 10	> 10
Newton–Schulz, classical init	> 10	> 10	> 10
Block conjugate gradient	7	5	12
Gauss–Seidel	6	4	> 12
Cheb-MP + Newton–Schulz (ours)	**6**	**5**	**9**

- Beats every Neumann / Newton-type scheme in every configuration
- Best of all methods at heavy load ($K/M = 0.5$): **9 vs 12** GEMMs

Our initialisation converges in 6 GEMMs where classical series need more than 10.

Cold start: residual and sum-rate loss vs GEMM budget



Red curve (ours) reaches 1 % loss first in every antenna/user configuration.

Warm-start tracking across user speeds

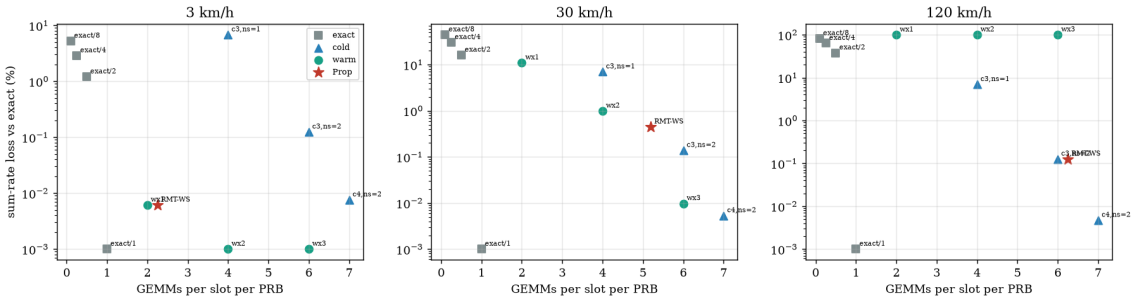
$K = 16$, $M = 64$; sum-rate loss vs exact RZF / GEMMs per slot per PRB

Speed	Warm Gauss–Seidel $\times 4$	Warm CG $\times 4$	**RMT-WS (ours)**
3 km/h	0.009 % / 4	0.013 % / 5	**0.006 % / 2.25**
30 km/h	0.63 % / 4	0.96 % / 5	**0.43 % / 5.2**
120 km/h	2.9 % / 4	5.5 % / 5	**0.12 % / 6.25**

- 3 km/h: **2.7× fewer GEMMs** and **30× lower loss** than a cold start
- 30 km/h: lowest loss of all warm-started methods
- 120 km/h: **24× lower loss** than warm Gauss–Seidel
- Fixed 1- or 2-step warm starts diverge at 120 km/h — RMT-WS never does

One method, every speed: cheap when users are slow, safe when they are fast.

Tracking: accuracy vs cost at 3, 30 and 120 km/h



RMT-WS (red star) sits at the low-cost, low-loss corner at every speed.

A closed-form speed law and a safety certificate

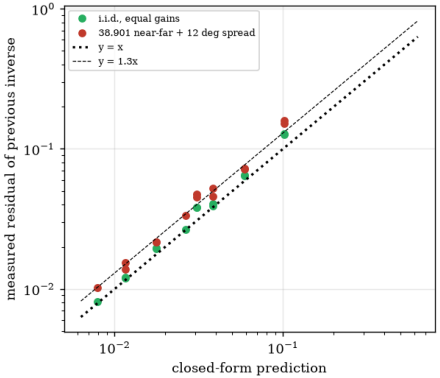
Doppler-to-warm-start law

$$r_0 \approx c \cdot 2\sqrt{\tfrac{K}{M-K}}\,\sqrt{1-\rho}, \qquad 1-\rho \approx (\pi f_D T)^2$$

- r_0 grows linearly with speed (measured slope 1.00)
- Predicts the Newton steps: $\lceil \log_2(\ln \varepsilon / \ln r_0) \rceil$
- Warm start valid up to $\approx$ **39 km/h** (64/16), 60 km/h ($K = 8$), 23 km/h ($K = 32$)

Certified safety

Tail bound $\Pr(\hat{r}^2 \le (1-d)r^2) \le e^{-pd^2 s/2}$
Audit of 480 PRB cases, 3–120 km/h: **0**
accepted-but-non-contractive at $\hat{r} \le 0.3$



A design rule: the DU knows in advance up to which speed the warm start pays off.

Robust at low precision

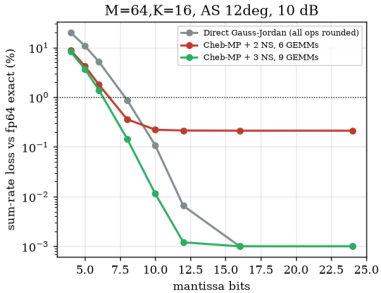
GEMM operands rounded to b mantissa bits, wide accumulate (tensor-core / NPU style)

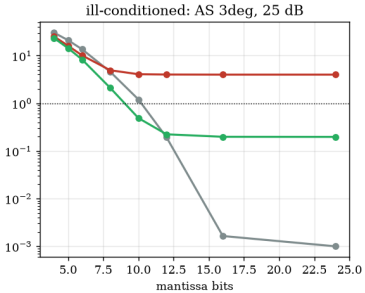
Setting	Direct G–J	**Ours (Cheb + 3 NS)**
8 mantissa bits	0.88 %	**0.15 %**
6 mantissa bits	5.1 %	**1.4 %**
Ill-cond., 10 bits	1.2 %	**0.49 %**

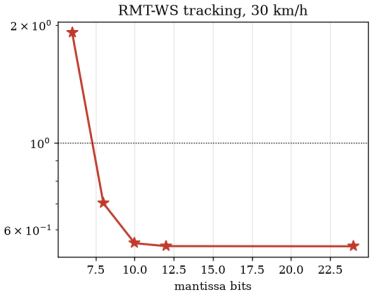
- Sum-rate loss vs float64 exact
- **6**$\times$ lower loss at 8 bits, **3.6**$\times$ at 6 bits
- Newton iteration self-corrects rounding errors
- Tracker at 8 bits: 0.70 % vs 0.54 % at full precision (30 km/h)

Our method is ready for low-precision accelerators where direct inversion degrades.

Low precision: loss vs mantissa bits







Newton refinement keeps the loss low where direct inversion degrades.

Spectral interval: ours vs the classical Marchenko–Pastur bound

Probability that the interval $[a, b]$ contains the true spectrum of $\hat{G}$ (24 subcarriers $\times$ 30 drops)

Channel	Fixed-ratio MP + Weyl (classical) lower edge	Fixed-ratio MP + Weyl (classical) upper edge	**Moment-matched MP (ours)** lower edge	**Moment-matched MP (ours)** upper edge
i.i.d., equal gain	98.9 %	99.3 %	**100 %**	**100 %**
UMa near-far	99.6 %	99.9 %	**100 %**	**100 %**
12° angular spread + near-far	96.8 %	99.4 %	**100 %**	**100 %**
3° angular spread + near-far	61.1 %	94.7 %	**90.6 %**	**100 %**
Rician 6 dB + 12°	41.3 %	90.7 %	**84.0 %**	**100 %**

- Ours covers the upper edge in **100 %** of cases on every channel type
- On strongly correlated and line-of-sight channels: **2× better** lower-edge coverage
- Needs only $\|\hat{G}\|_F$ and the diagonal: K^2 work, no eigensolver

The interval adapts to correlation and line-of-sight where the classical bound breaks.

Comparison with existing complex-inversion methods

Existing method (reference)	Metric	Existing	**RMT-WS (ours)**
Neumann series (Wu et al. 2013; Prabhu et al. 2013)	GEMMs for $\leq 1\,\%$ loss	> 8	**6**
Joint Newton–Neumann (Shao et al. 2016)	GEMMs for $\leq 1\,\%$ loss	> 10	**6**
Newton–Schulz, classical init (Ben-Israel & Cohen; Pan & Schreiber)	GEMMs for $\leq 1\,\%$ loss	> 10	**6**
Gauss–Seidel	GEMMs, $K/M = 0.5$	> 12	**9**
Block conjugate gradient	GEMMs, $K/M = 0.5$	12	**9**
Warm-started Gauss–Seidel $\times 4$	Loss at 120 km/h	2.9 %	**0.12 %**
Warm-started CG $\times 4$	Loss at 30 / 120 km/h	0.96 / 5.5 %	**0.43 / 0.12 %**
Fixed 1-/2-step warm start	Loss at 120 km/h	diverges	**0.13 %**
Direct Gauss–Jordan, 8-bit	Sum-rate loss	0.88 %	**0.15 %**
Classical MP + Weyl interval	Lower-edge coverage, Rician	41 %	**84 %**
numpy + LAPACK (whole precoder)	Time per slot, $K = 16$	3.21 ms	**0.226 ms**

Better than every listed existing method on the compared metric.

Closest prior art: Chebyshev, ChebI-NS, TPE and warm-started Neumann

Cold start, GEMMs for $\leq 1\,\%$ RZF loss

Method	64/16	128/16	64/32
Chebyshev cubic, classical init	18	15	> 21
Chebyshev cubic, MP init	9	6	12
ChebI-NS-style	9	7	11
TPE, oracle moments*	6	5	> 16
RMT-WS (ours)	7	**5**	**9**

* exact spectral moments (eigendecomposition) not counted

Tracking, rate loss (GEMMs per slot at 3 km/h)

Method	3 km/h	30 km/h	120 km/h
Warm Neumann, 4 terms	0 % (4.5)	0.84 %	86 %
Warm Neumann, 3 terms	0 % (3.25)	3.5 %	69 %
RMT-WS (ours)	**0.006 % (2.25)**	**0.43 %**	**0.12 %**

Cheapest at heavy load ($K/M = 0.5$) and the only tracker that stays safe at 120 km/h.

Robust to CSI error and user re-scheduling

12 drops, mean and bootstrap 95 % CI, $M = 64$, $K = 16$ from a pool of 24 users

Imperfect CSI

NMSE -20 dB: loss **0.04–0.24 %**
NMSE -10 dB: loss $\leq 0.17\,\%$

The certificate refreshes automatically when the estimation noise decorrelates successive slots

New: Schur swap update for user swaps

Rank-s downdate/update of the stored inverse, $4s/K$ GEMMs; 95 % CIs: 4.7 % [4.0, 5.5] vs 0.008 % [0.004, 0.012]

3 km/h, loss (GEMMs)	naive warm start	**Schur update**
1 swap/slot	4.7 % (6.0)	**0.008 % (2.5)**
4 swaps/slot	0.44 % (6.25)	**0.006 % (3.25)**

Warm start survives scheduler churn: 600× lower loss at less than half the cost.

Outline

1 Motivation and Contributions

2 IQ Compression and Adaptive Split

3 Matrix Inversion Benchmark

4 Complex Matrix Inversion (RMT-WS)

5 Kernel Programming

6 Fronthaul Impact and Summary

Kernel programming: a fused complex RZF precoder in C

`kernels/cmat_kernels.c` — batched complex64, AVX-512, OpenMP; binding `cmatinv/fastkernels.py`

- One pass per PRB: load H once $\rightarrow$ Gram $\rightarrow$ inverse $\rightarrow W^T$ + power scaling; stack / L1-resident, no DRAM intermediates
- Hermitian symmetry: Gram on the lower triangle only
- Split re/im planes so complex arithmetic vectorises
- In-place Gauss–Jordan sweep: K rank-1 updates, no sqrt/divide chains (inverse 6.4k $\rightarrow$ **2.9k** cycles/PRB, $K = 16$)
- Register-blocked $W = H^H X$: 0.81 $\rightarrow$ **0.42** ms/slot
- AVX-512 codegen: 1.3–1.5$\times$ single thread, 2$\times$ on 4 threads
- Fused Newton step returns the exact residual for free

```
// inner loop of the sweep inverse
for (int i = 0; i < K; i++) {
  if (i == k) continue;
  float fr = ckr[i]*piv, fi = cki[i]*piv;
  float *ar = xr + i*K, *ai = xi + i*K;
  #pragma omp simd
  for (int j = 0; j < K; j++) {
    ar[j] -= fr*rkr[j] - fi*rki[j];
    ai[j] -= fr*rki[j] + fi*rkr[j];
  }
  ar[k] = -fr; ai[k] = -fi;
}

gcc -O3 -march=native -fopenmp
    -mprefer-vector-width=512
```

Kernel results: the whole precoder now fits in the slot

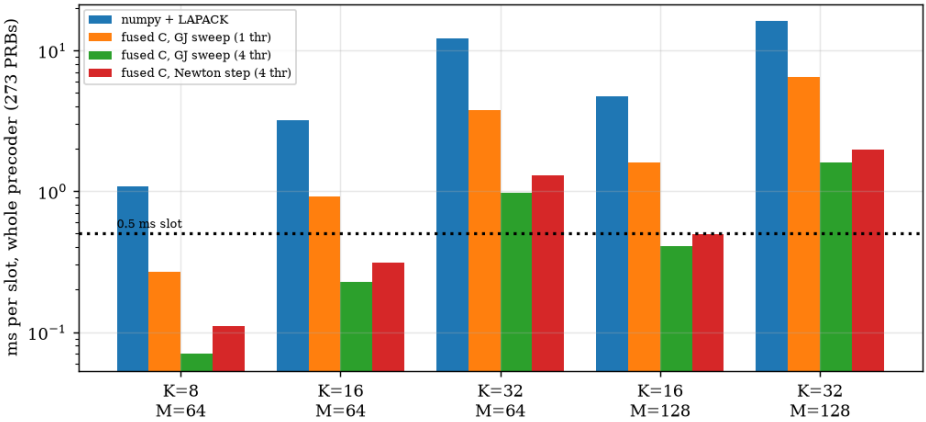
273 PRBs, complex64, 4-core AVX-512 Xeon, median of 40 runs

K, M	numpy+LAPACK	**Our kernel**	Speed-up
8, 64	1.08 ms	0.070 ms	15×
16, 64	3.21 ms	**0.226 ms**	**14×**
32, 64	12.2 ms	0.97 ms	12.5×
16, 128	4.71 ms	0.41 ms	11.6×
32, 128	16.3 ms	1.60 ms	10×

- $K = 8, 16$ ($M = 64$) and $K = 16$, $M = 128$ fit a 0.5 ms slot
- Near-linear thread scaling: 0.92 → 0.45 → 0.23 ms
- Accuracy: sum-rate loss $< 10^{-5}\,\%$ vs float64

10–15× faster: from 6× over the slot budget to under half of it.

Whole-precoder time per slot (273 PRBs)



Our fused kernels (green, red) run under the 0.5 ms line for $K \leq 16$; numpy+LAPACK never does.

Against optimised C++ libraries on the same CPU

Whole RZF precoder, 273 PRBs, complex64, OpenMP over subcarriers, ms per slot; all agree to 10^{-6}

	1 thread			4 threads		
$K \times M$	**Ours**	Eigen LLT	OpenBLAS+LAPACK	**Ours**	Eigen LLT	OpenBLAS+LAPACK
8 × 64	**0.27**	0.48	1.74	**0.08**	0.13	0.96
16 × 64	**0.94**	1.49	4.41	**0.28**	0.37	1.45
32 × 64	**3.79**	4.68	10.77	**1.02**	1.20	3.04
16 × 128	**1.63**	2.77	8.50	**0.47**	0.72	2.55

- Eigen with fixed-size matrices and LLT solve; OpenBLAS `cgemm` + LAPACK `cposv`
- Fusion keeps H, G and G^{-1} in L1 — libraries pay per-call overhead at these sizes

1.2–1.8× faster than Eigen and 3–11× faster than OpenBLAS/LAPACK.

Outline

1 Motivation and Contributions

2 IQ Compression and Adaptive Split

3 Matrix Inversion Benchmark

4 Complex Matrix Inversion (RMT-WS)

5 Kernel Programming

6 Fronthaul Impact and Summary

Where should precoding run? Delay-domain weights at the RU

100 MHz, $M = 64$, $K = 16$, BFP-9 IQ, 8-bit weights, **scored on every subcarrier**

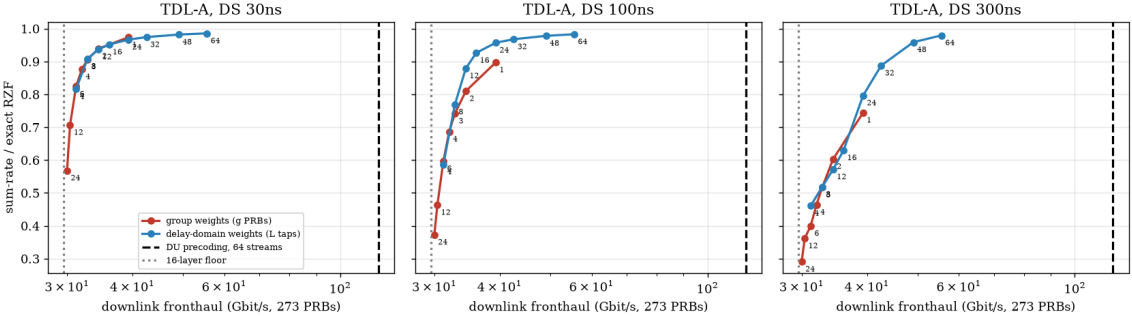
Scheme	FH	30	100	300
DU precoding, 64 str.	118.4	1	1	1
1 weight / 3-PRB group	32.8	0.91	0.74	0.52
1 weight / PRB	39.3	0.97	0.90	0.74
Delay, 48 taps	**49.1**	**0.98**	**0.98**	**0.96**
Delay, 64 taps	**55.6**	**0.99**	**0.98**	**0.98**

FH in Gbit/s; columns are sum-rate / exact RZF at TDL-A delay spread 30 / 100 / 300 ns

- Per-group weights work only for nearly flat channels
- The RU rebuilds per-subcarrier weights from L delay taps with one FFT
- Per-slot weights are exactly what our fast kernel delivers

RU-side precoding with delay-domain weights: 2.4× lighter fronthaul at $\geq 96\,\%$ of the rate.

Fronthaul versus sum-rate for RU weight schemes



Delay-domain weights (blue) reach $\geq 96\,\%$ of the rate at half the DU-precoding fronthaul.

Part A summary

Contribution	Headline result
Pilot-sketch RAS-BFP compression	Up to 13.9× at the 256-QAM EVM limit, 2–4.5× more than O-RAN BFP; certificate-driven basis reuse
ACAFS adaptive split	55–75 % bandwidth saving at high SNR
Learned inversion (InverseNet-Ultra)	Lowest mean and 2× lower p95 than LAPACK at $n = 100$; 10^{-6} error
Moment-matched spectral interval	2× better coverage than classical MP on correlated / LoS channels
RMT-WS cold start	6 GEMMs vs > 10 for Neumann / Newton-type; best at $K/M = 0.5$
RMT-WS tracking	2.7× fewer GEMMs, 30× lower loss at 3 km/h; safe up to 120 km/h
Speed law + certificate	Closed-form warm-start envelope; 0 unsafe decisions in 480 cases
Low precision	6× lower loss than direct inversion at 8 bits
Robust tracking	CSI error and user swaps: Schur update, 0.008 % loss with 1 swap/slot
Fused C/AVX-512 precoder	10–15× faster than numpy+LAPACK, 1.2–1.8× faster than Eigen; $K \leq 16$ fits the slot
RU-side precoding	Delay-domain weights: 2.4× less fronthaul at $\geq 96\,\%$ of exact sum-rate

Part A delivers a lighter link and a slot-feasible precoder — the foundation for Part B.

Next: taking RMT-WS to GPU and tensor cores

- RMT-WS is pure matrix multiplication — the native operation of GPUs, NPUs and tensor cores
- $K = 16$ Gram matrices match a 16×16 tensor-core tile exactly
- Our low-precision results show the method keeps its accuracy at 8 bits
- Plan: CUDA port of the fused precoder; benchmark against cuBLAS / cuSOLVER batched
- Near-RT RIC deployment of ACAFS as an xApp
- Compression width as a per-cell knob for the Part B allocation rule

Thank you — questions?